

DOCUMENTO DE ENGENHARIA DE SOFTWARE

Sistema Hospitalar para Agendamento de Consultas e
Gerenciamento de Histórico Médico

Projeto: Tech Challenge – Fase 3

Data: Agosto de 2026

Sistema Hospitalar — Backend Modular em Microserviços

Sumário

1. Introdução	3
1.1 Contextualização	3
1.2 Objetivo	3
1.3 Escopo	3
2. Processo de Desenvolvimento	3
2.1 Metodologia	3
2.2 Arquitetura	4
2.3 Tecnologias	4
2.4 Tecnologias Utilizadas	5
2.5 Divisão das Atividades da Equipe	6
3. Requisitos	7
3.1 Requisitos Funcionais	7
3.2 Requisitos Não Funcionais	7
4. Técnicas de Levantamento de Requisitos	7
5. UML – Introdução	8
6. Diagrama de Casos de Uso	8
7. Documentação dos Casos de Uso	10
UC01 – Autenticar Usuário	10
UC02 – Registrar Consulta	10
UC03 – Editar Consulta	10
UC04 – Consultar Histórico	10
UC05 – Visualizar Consultas	11
UC06 – Enviar Lembrete	11
8. Considerações Finais	11

1. Introdução

1.1 Contextualização

O setor da saúde exige sistemas confiáveis, capazes de organizar consultas médicas, armazenar históricos clínicos e permitir comunicação eficiente entre profissionais e pacientes. Além disso, a segurança das informações é indispensável em razão da natureza sensível dos dados médicos.

O Tech Challenge propõe o desenvolvimento de um backend para um sistema hospitalar modular, utilizando autenticação, autorização, comunicação assíncrona e arquitetura baseada em serviços. O sistema deve atender a diferentes perfis de usuários, garantindo que cada um tenha acesso apenas às funcionalidades permitidas.

1.2 Objetivo

Desenvolver uma aplicação backend segura, escalável e modular que permita:

- Gerenciamento de consultas médicas;
- Consulta ao histórico dos pacientes;
- Envio automático de lembretes;
- Autenticação e autorização dos usuários;
- Comunicação assíncrona entre serviços.

1.3 Escopo

O sistema contempla:

- Cadastro de consultas;
- Alteração de consultas;
- Histórico médico;
- Consulta via GraphQL;
- Controle de acesso por perfil;
- Comunicação entre microserviços utilizando RabbitMQ ou Kafka;
- Serviço de notificações.

2. Processo de Desenvolvimento

2.1 Metodologia

Foi adotada uma metodologia ágil baseada em Scrum.

Papéis

- Product Owner;
- Scrum Master;
- Equipe de Desenvolvimento.

Cerimônias

- Sprint Planning;
- Daily Meeting;
- Sprint Review;
- Sprint Retrospective.

2.2 Arquitetura

A solução foi estruturada em microsserviços, conforme descrito a seguir.

Serviço de Agendamento

Responsável por:

- Criar consultas;
- Editar consultas;
- Publicar eventos para notificação.

Serviço de Notificações

Responsável por:

- Consumir mensagens da fila;
- Enviar lembretes aos pacientes.

Serviço de Histórico (Opcional)

Responsável por:

- Armazenar histórico médico;
- Disponibilizar consultas por meio do GraphQL.

Essa separação favorece a escalabilidade, o baixo acoplamento e a facilidade de manutenção da solução.

2.3 Tecnologias

Stack tecnológica principal adotada no projeto:

- Java
- Spring Boot
- Spring Security
- GraphQL
- RabbitMQ ou Kafka
- PostgreSQL
- Maven
- Docker
- GitHub

2.4 Tecnologias Utilizadas

Para o desenvolvimento do sistema hospitalar serão utilizadas tecnologias modernas, amplamente empregadas em aplicações corporativas, garantindo segurança, escalabilidade, manutenibilidade e alto desempenho.

Tecnologia	Finalidade
Java 21	Linguagem principal utilizada no desenvolvimento do backend.
Spring Boot	Framework para construção da API REST e configuração da aplicação.
Spring Security	Autenticação e autorização baseadas em perfis de usuários (Médico, Enfermeiro e Paciente).
Spring Data JPA	Camada de persistência responsável pela comunicação com o banco de dados.
Hibernate	Framework ORM utilizado pelo Spring Data JPA para mapear objetos em tabelas relacionais.
GraphQL	Consulta flexível ao histórico de consultas e informações dos pacientes.
RabbitMQ (ou Kafka)	Comunicação assíncrona entre os microsserviços, permitindo o envio de eventos ao serviço de notificações.
PostgreSQL	Banco de dados relacional utilizado para armazenar usuários, consultas e histórico médico.
Docker	Containerização da aplicação e dos serviços, facilitando implantação e execução.
Maven	Gerenciamento de dependências e automação do processo de build.
JUnit 5	Desenvolvimento de testes unitários.
Mockito	Criação de objetos simulados (mocks) para testes unitários.
Postman	Testes e validação dos endpoints da API e das consultas GraphQL.
Git	Controle de versão do código-fonte.
GitHub	Hospedagem do repositório e colaboração entre os integrantes da equipe.
Swagger / OpenAPI	Documentação interativa dos endpoints da API REST.

2.5 Divisão das Atividades da Equipe

O desenvolvimento do projeto será realizado por uma equipe composta por cinco integrantes. A divisão das atividades foi planejada para distribuir as responsabilidades de forma equilibrada, permitindo que cada integrante atue em uma área específica do sistema e contribua para a integração final da solução.

Integrante	Responsabilidade	Atividades
Integrante 1	Líder Técnico / Arquiteto	Definição da arquitetura, configuração inicial do projeto, organização do repositório, revisão de código, integração dos microsserviços e apoio técnico à equipe.
Integrante 2	Backend – Agendamento	Desenvolvimento das funcionalidades de cadastro, edição, cancelamento e consulta de consultas; criação dos endpoints REST e integração com o banco de dados.
Integrante 3	Backend – Segurança e GraphQL	Implementação do Spring Security, autenticação, autorização por perfis (Médico, Enfermeiro e Paciente) e desenvolvimento das consultas GraphQL para o histórico médico.
Integrante 4	Microsserviços e Mensageria	Implementação do RabbitMQ, desenvolvimento do Serviço de Notificações, publicação e consumo de eventos, além da comunicação assíncrona entre os serviços.
Integrante 5	Qualidade e Documentação	Desenvolvimento dos testes unitários e de integração, criação da documentação técnica (Swagger), elaboração da collection do Postman, documentação de Engenharia de Software e apoio na validação final do projeto.

3. Requisitos

3.1 Requisitos Funcionais

Código	Descrição
RF01	O sistema deverá permitir a autenticação de usuários.
RF02	O sistema deverá controlar permissões conforme o perfil do usuário.
RF03	Médicos poderão visualizar e editar históricos médicos.
RF04	Enfermeiros poderão registrar consultas.
RF05	Pacientes poderão visualizar apenas suas próprias consultas.
RF06	O sistema deverá permitir consultas utilizando GraphQL.
RF07	O sistema deverá permitir o cadastro de consultas.
RF08	O sistema deverá permitir a edição de consultas.
RF09	Após o cadastro de uma consulta, deverá ser enviada uma mensagem ao serviço de notificações.
RF10	O serviço de notificações deverá enviar lembretes automaticamente.

Esses requisitos refletem diretamente as necessidades apresentadas no desafio.

3.2 Requisitos Não Funcionais

Código	Descrição
RNF01	A autenticação deverá utilizar Spring Security.
RNF02	A comunicação entre serviços deverá ocorrer de forma assíncrona.
RNF03	O sistema deverá possuir arquitetura modular.
RNF04	Os serviços deverão ser desacoplados entre si.
RNF05	O código deverá seguir boas práticas de organização.
RNF06	A documentação deverá conter os endpoints e as instruções de execução.

Esses requisitos estão alinhados aos critérios de qualidade e documentação exigidos pelo Tech Challenge.

4. Técnicas de Levantamento de Requisitos

Embora o desafio original não especifique uma técnica de elicitação, recomenda-se, para este projeto, a aplicação das seguintes práticas de Engenharia de Software:

Entrevistas

Realizadas com médicos, enfermeiros, administradores e pacientes, com o objetivo de compreender as necessidades de cada perfil.

Observação

Análise de como ocorre atualmente o processo de agendamento das consultas.

Brainstorming

Reuniões entre a equipe técnica e especialistas do hospital para identificar oportunidades de melhoria.

Análise Documental

Consulta às normas internas do hospital e aos requisitos legais referentes à proteção de dados dos pacientes.

Protótipos

Construção de protótipos das APIs e dos fluxos de atendimento para validação dos requisitos junto aos usuários.

5. UML – Introdução

A UML (Unified Modeling Language) é uma linguagem padrão para modelagem de sistemas orientados a objetos. Seu objetivo é representar visualmente os elementos do software antes da implementação.

Para este projeto, os principais diagramas que poderiam ser utilizados são:

- Casos de Uso;
- Classes;
- Sequência;
- Componentes;
- Implantação.

Neste documento é detalhado o Diagrama de Casos de Uso.

6. Diagrama de Casos de Uso

Atores

- Médico;
- Enfermeiro;
- Paciente;
- Sistema de Notificações.

Casos de Uso

- Autenticar usuário;
- Registrar consulta;
- Editar consulta;
- Consultar histórico;
- Consultar próximas consultas;

- Visualizar consultas;
- Enviar lembrete.

Relação Ator × Caso de Uso

Ator	Caso de Uso
Médico	Registrar Consulta
Médico	Editar Consulta
Médico	Consultar Histórico
Enfermeiro	Registrar Consulta
Enfermeiro	Consultar Histórico
Paciente	Visualizar Consultas
Paciente	Consultar Próximas Consultas
Sistema de Notificações	Enviar Lembretes

7. Documentação dos Casos de Uso

UC01 – Autenticar Usuário

Objetivo: permitir o acesso seguro ao sistema.

Atores: Médico, Enfermeiro e Paciente.

Pré-condição: usuário previamente cadastrado.

Fluxo Principal

1. Informar usuário.
2. Informar senha.
3. Validar credenciais.
4. Liberar acesso.

Pós-condição: usuário autenticado.

UC02 – Registrar Consulta

Ator: Médico ou Enfermeiro.

Fluxo

5. Selecionar paciente.
6. Informar data.
7. Informar horário.
8. Salvar consulta.
9. Publicar evento para RabbitMQ/Kafka.

Pós-condição: consulta cadastrada.

UC03 – Editar Consulta

Ator: Médico.

Fluxo

10. Localizar consulta.
11. Alterar dados.
12. Salvar alterações.
13. Publicar evento para notificações.

UC04 – Consultar Histórico

Atores: Médico e Enfermeiro.

Fluxo

14. Selecionar paciente.

15.Sistema consulta o histórico via GraphQL.

16.Histórico é retornado.

UC05 – Visualizar Consultas

Ator: Paciente.

Fluxo

17.Login.

18.Consultar agenda.

19.Sistema retorna apenas as consultas do paciente.

UC06 – Enviar Lembrete

Ator: Sistema de Notificações.

Fluxo

20.Receber mensagem.

21.Processar evento.

22.Identificar paciente.

23.Envia lembrete automaticamente.

8. Considerações Finais

O sistema hospitalar proposto atende aos requisitos apresentados no Tech Challenge ao utilizar autenticação com Spring Security, controle de acesso baseado em perfis, consultas por GraphQL, comunicação assíncrona por RabbitMQ ou Kafka e arquitetura modular baseada em microsserviços.

A documentação desenvolvida aplica práticas de Engenharia de Software para organizar os requisitos, descrever os processos de desenvolvimento e especificar os principais casos de uso, facilitando a implementação, a manutenção e a evolução do sistema.

As informações específicas do desafio foram extraídas do material fornecido, enquanto a estrutura de engenharia foi elaborada para atender aos padrões acadêmicos de documentação.

Documento elaborado para o Tech Challenge – Fase 3 – Engenharia de Software